Systems programming lesson 4

COM312: BSc Computer science

Topics area coverage.

Process management and scheduling
• Process identifications • Process trees, chains, and fans • Zombies and ophan processes • Process groups, sessions and controlling terminals • Unix daemons • Systems vR3 and systemVR4 scheduler

Process management and scheduling

- The most fundamental concept in modern OS is the **process.**
- The principle function of the OS is to **create, manage,** and **terminate processes**.
- To perform its process management functions, the OS maintains a description of each process, or process image.
- During its lifetime, a process moves among a number of states. The most of important of these are **Ready, Running, and Blocked**

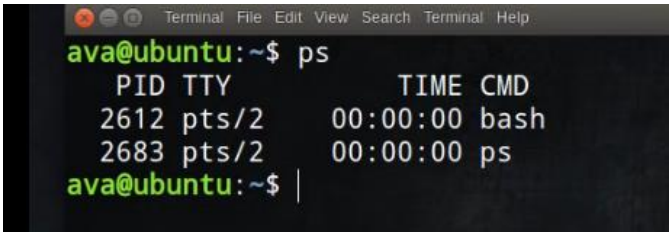
Operations on Processes

The processes in the systemexecute concurrently and they must be **created** and **deleted** dynamically thus the operating system must provide the mechanism for the creation and deletion of processes.

Program vs. Process

Definition: a **process** is the OS *representation of a program during its execution*

How we check the processes. using the ps command



Program vs process

Lets look at the difference between process and program

Process	Program
The process is basically an instance of the computer program that is being executed.	A Program is basically a collection of instructions that mainly performs a specific task when executed by the computer.
A process has a **shorter lifetime**.	A Program has a **longer lifetime**.
A Process requires resources such as memory, CPU, Input-Output devices.	A Program is stored by hard-disk and does not require any resources.
A process has a dynamic instance of code and data	A Program has static code and static data.
Basically, a process is the **running instance** of the code.	On the other hand, the program is the **executable code**.

- Why?
  - A user program is something static: *it is just a sequence of bytes stored in a "disk"*

- When user asks the O.S. for executing a program, in a multiprogrammed-multiuser environment, the system needs to represent that particular "execution of X" with some attributes
  - Which regions of physical memory is using
  - Which files is accessing
  - Which user is executing it
  - What time it is started
  - How many CPU time it has consumed?

- Every process in a UNIX system has the following attributes:

- **User program** the program to be executed
- **some user data** – the modifiable part of the user space. May include program data, user stack area, and programs that may be modified
- **a stack** – each stack has one or more last-in-first-out (**LIFO**) stacks associated with it. A stack is used to store parameters and calling addresses for procedure and system calls.

- **Process control Block-** Data needed by the OS to control the process
- a unique process ID number (PID)

process states

- As a process executes, it changes state.
- The state of a process is defined in part by the current activity of that process.

Each process may be in one of the following states;

i. **NEW**–the process is being created
ii. **RUNNING**–instructions are being executed
iii. **WAITING**- the process is waiting for some event to occur (such as an I/O completion or reception of a signal)
iv. **READY**- the process is waiting to be assigned to a processor
v. **TERMINATED**- the process has finished execution

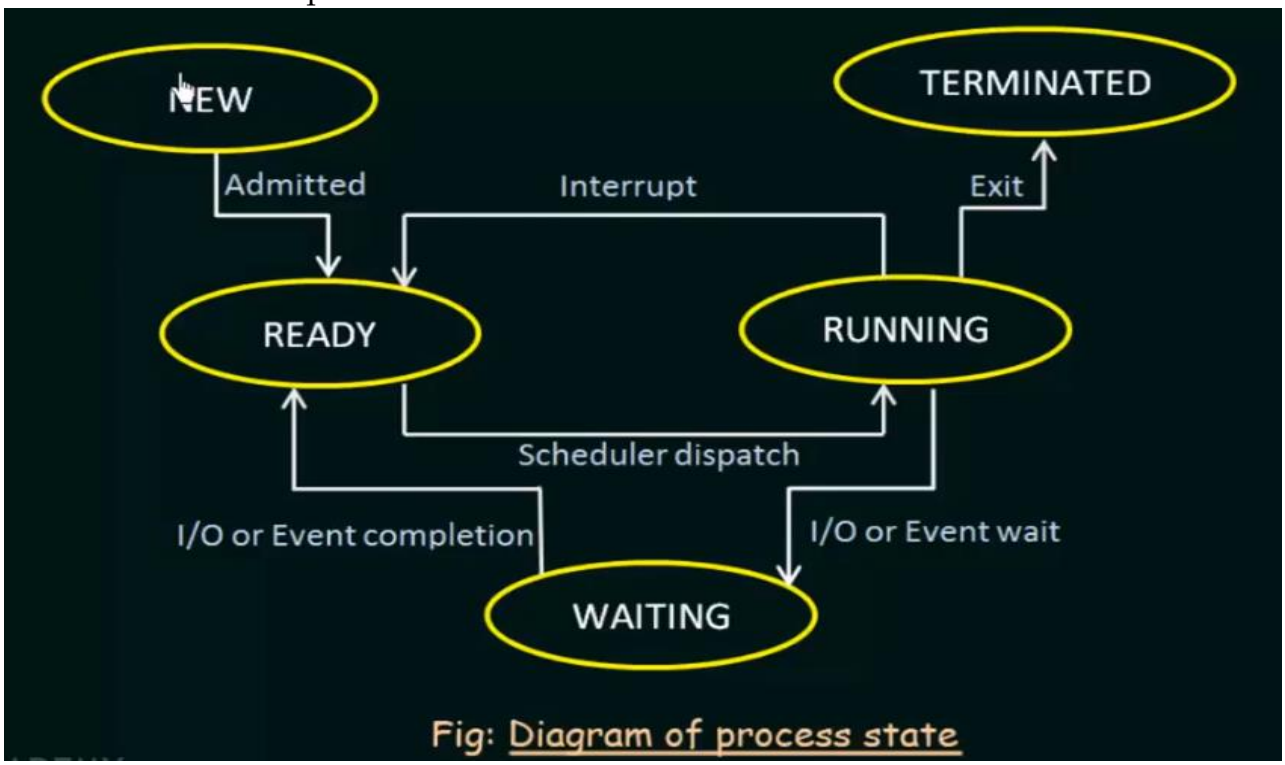


Fig: Diagram of process state

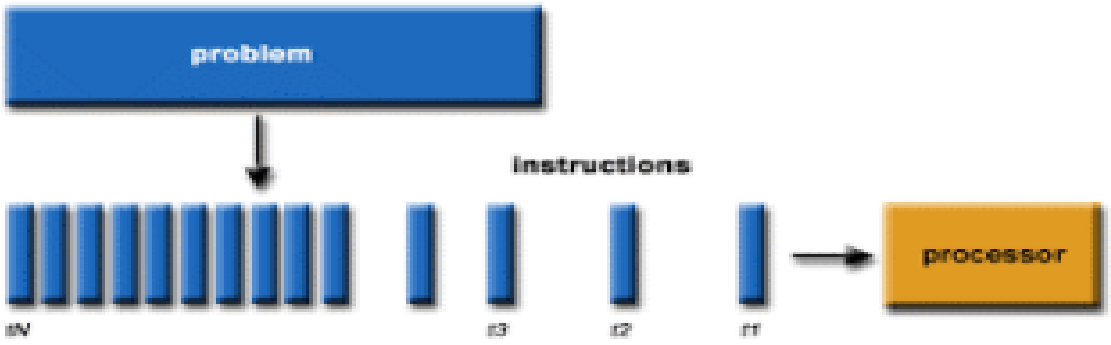
The Basic process creation using fork(), exec() wait(), exit() used in system call in Linux processes

FORK() – creates a new process
EXEC() – replaces current process image with a new process image
WAIT() – used parent process to wait for child's completion
EXIT() – used to terminate a process.

FORK & EXEC in LINUX

The main aim of fork() is for development of parallel computing

- **Parallel computing** refers to the process of executing several processors **an application or computation simultaneously.** Generally, it is a kind of computing architecture where the large problems break into independent, smaller, usually similar parts that can be processed in one go.



What is the purpose of folk?

- System call fork() is **used to create processes**.
- It takes no arguments and returns a process ID.
- The purpose of fork() is to create a new process, which becomes the child process of the caller. After a new child process is created, both processes will execute the next instruction following the fork() system call

Why does fork() return twice?

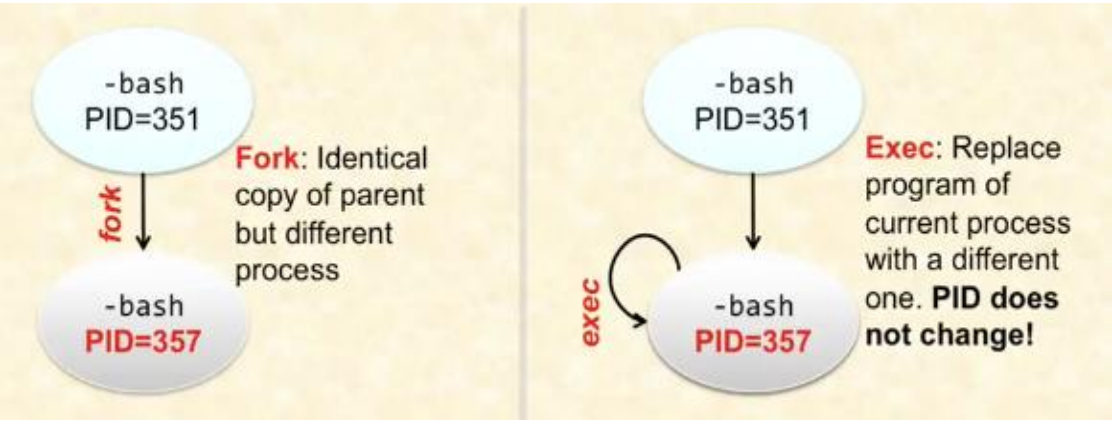
Once within the parent process and once within the child ... **The OS arranges for the fork() to return in the parent process with the pid of the child**, and for it to return in the child process with zero (**if things go well**). That is why they say that fork() returns twice.

- **Running a new program on Linux involves 2 steps.**

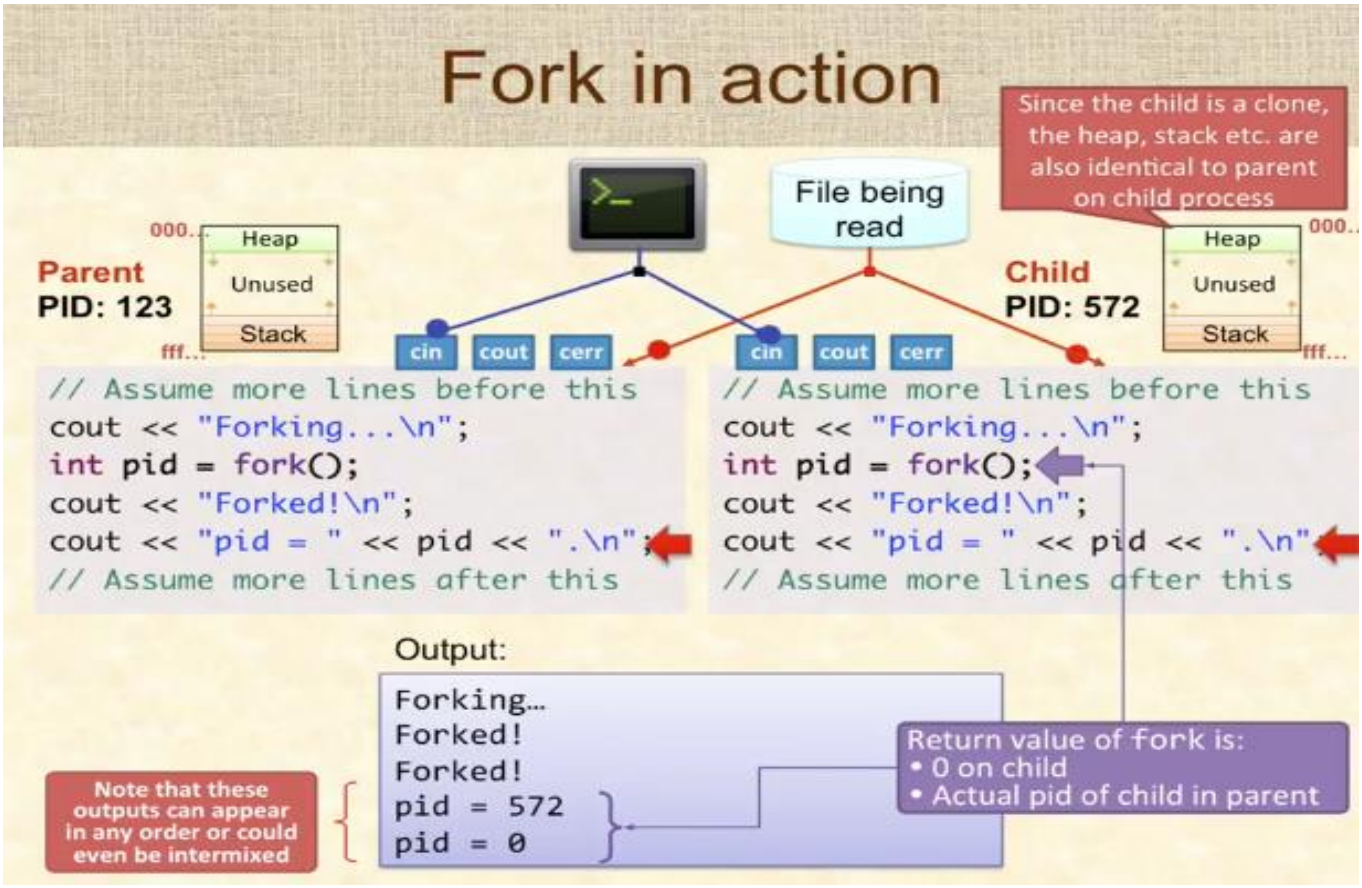
1. **FORK**: Clone current process
   - Fork() system call is used to create a separate, duplicate process.

- The fork() system call **allows one process, the parent, to create a new process, the child**. This is done by making the new child process an (almost) exact duplicate of the parent: the child obtains copies of the parent's *stack, data, heap, and text segments.* The term fork derives from the fact that we can envisage the parent process as dividing to yield two copies of itself

2. **EXEC**: Replace current program with a different one.
   - When an exec() system call is invoked, the program specified in the parameter to exec() will replace the entire process- including all threads.

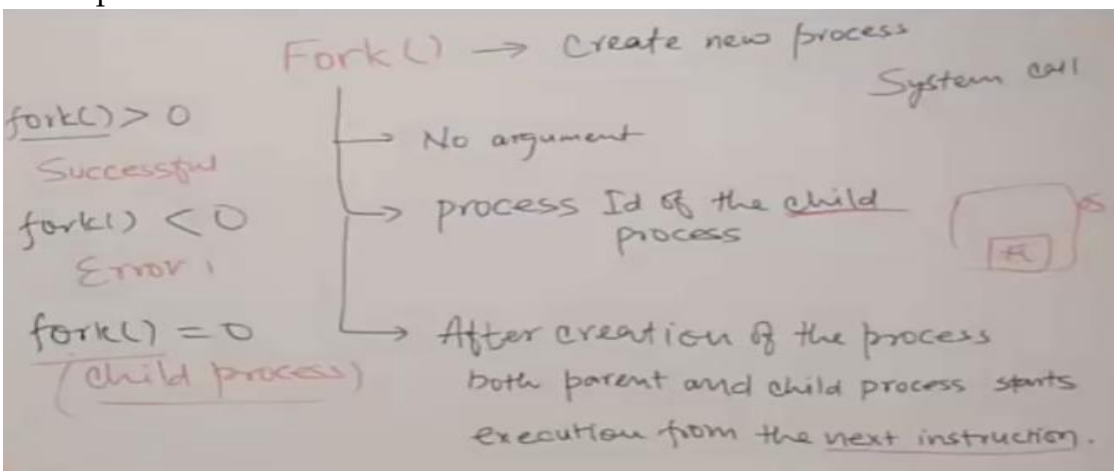


- **Fork is a system call used to clone a running process**
  - The cloned process is a child of the parent.
- **The parent and child are identical processes**
  - Have exactly the same stack (sequence of function calls)
  - Have same virtual memory (so pointers and other data is preserved)
  - Have same set of files and consequently have
    - Same set of open files, devices, sockets, etc.
- **The cloned child differs from the parent in:**
  - The process ID (pid)
  - The returned value from fork in the parent and child a slightly different
    - This is used to tell the difference between **parent** and **child** process



Code explained

- The program is running in memory
- Reads data from more files
- Calls fork() system call and creates new program identical copy to that of the parent.



Example Question

After a program executes the following series of fork() calls, how many new processes will result (assuming that none of the calls fails)?

fork();

fork();

fork();

Result

The first fork() call creates one new child. Both parent and child carry on to execute the second fork(), and thus each creates a further process, making a total of four processes. All four processes carry on to execute the next fork(), each creating a further child. Consequently, a total of seven new processes are created.

Forking processes has advantages

- **Threads do not enjoy many of these advantages**

1. **Fault tolerance**: if child process crashes parent process is unaffected heavily used in web-servers and databases to improve resilience
2. **Security**: parent and child can have different security settings (or even run as different users) for better auditing,
3. **Low latency**: since memory and I/O streams are identical start-up overheads are minimized
4. **Concurrency & performance**: parent and child can collaborate to perform different operations to improve performance
5. **Resource utilization**: processes get full set of resources enabling effective utilization of computational platforms
   - Easier to migrate processes across cloud infrastructures
   -

Zombies and orphan processes

What is Linux zombie process?

Zombie processes in Linux are sometimes also referred to as defunct or dead processes. They're **processes that have completed their execution**, but their entries are not removed from the process table. hasn't been properly cleaned up.

- A zombie is created by programs that crash or weren't adequately coded to handle the cleanup of a crash or exit.

- Technically speaking, when a process dies, the process descriptor remains in memory and becomes labeled EXIT_ZOMBIE. At this point, the parent process is notified that the child process has died with the SIGCHLD signal and the parent is supposed to execute the wait system call, which allows the parent to collect information from the now-dead process. This generally happens almost immediately. But not always.

Issue the command ps aux | grep 'Z and if any zombie processes are hanging around, you should see them. Those zombie processes will be listed with an associated Process ID.

To kill a zombie process, you can send the SIGCHLD signal with the command kill -s SIGCHLD PID(Where PID is the process ID of the zombie process).

Typically, you should don't see any zombie processes on your system. But if your system is behaving oddly (not as in trying to chomp your brains), check for zombie processes and, if you find any, kill them immediately.

Example illustrating Zombie process

Zombie step 1 .....we start from fork() process

```
#include<stdio.h>
#include<stdlib.h>
int main()
{
      int p;
      p=fork();
      if(p>0)
       {
        printf("It's a parent process:%d",p);
       }
      else
       {
       printf("Child Process:%d",p);
       }
      return 0;
```

Code Explained

- First we examine the fork system call where by the parent will return positive number **while** child if successful will return zero value.

Zombie step 2 ..how to remove zombie from process table by use of wait () function

```c
#include<stdio.h>
#include<stdlib.h>
int main()
 {
     int p;
     p=fork();
     if(p>0)
      {
       printf("It's a parent process:%d",p);
       wait();
      }
     else
      {
      printf("Child Process:%d",p);
      }
     return 0;
 }
```

Code explanation

Using wait () function

- **Child process** will report to **parent process, then** that process will be removed from **the process table**

Zombie step 3 ... how to create a zombie process?

```c
#include<stdio.h>
#include<stdlib.h>
int main()
 {
     int p;
     p=fork();
     if(p>0)
      {
       printf("It's a parent process:%d",p);
       sleep(60);
      }
     else
      {
      printf("Child Process:%d",p);
      }
     return 0;
 }
```

Code explanation

Using sleep () function

- As long as **wait () function** is not called child process will terminate and will remain in the process table in that duration **Known as Zombie process**

Command to print all the process

```
ubuntu:~$ ps -e -o pid,stat,cmd
```

Executed program to illustrate Zombie process …

```
jgyan@ubuntu:~/Desktop$ gcc com.c
jgyan@ubuntu:~/Desktop$ ./a.out
Child Process:0^C
jgyan@ubuntu:~/Desktop$ gcc com.c
jgyan@ubuntu:~/Desktop$ ./a.out
Child Process:0
```

```
2849 Sl   /usr/lib/gvfs/gvfsd-metadata
2893 Sl   update-notifier
2900 Sl   gedit /home/jgyan/Desktop/com.c
2928 SNl  /usr/bin/python3 /usr/bin/update-manager --no-update
2953 Sl   /usr/lib/x86_64-linux-gnu/deja-dup/deja-dup-monitor
2964 Sl   gnome-terminal
2975 S    gnome-pty-helper
2976 Ss   bash
3040 S    [kworker/u128:1]
3069 S    /sbin/dhclient -d -sf /usr/lib/NetworkManager/nm-dhc
3427 Ss   /usr/sbin/cupsd -f
3430 S    /usr/lib/cups/notifier/dbus dbus://
3479 Sl   /usr/lib/x86_64-linux-gnu/unity-scope-home/unity-sco
3493 Sl   /usr/bin/unity-scope-loader applications/application
3495 Sl   /usr/lib/x86_64-linux-gnu/unity-lens-files/unity-fil
3518 Sl   /usr/lib/x86_64-linux-gnu/unity-lens-music/unity-mus
3551 Ss   bash
3735 S    [kworker/u128:0]
3750 T    ./a.out
3796 S    [kworker/u128:2]
3802 S+   ./a.out
3803 Z+   [a.out] <defunct>
3804 R+   ps -e -o pid,stat,cmd
jgyan@ubuntu:~$
```

Code output

Z + a.out child process, process waiting to report to the parent. But parent hasn't called **wait () function**. That why child process is z+ (zombie)

If you delete sleep(60) function? What will happen?

```
3479 Sl   /usr/lib/x86_64-linux-gnu/unity-scope-home/unity-sco
3493 Sl   /usr/bin/unity-scope-loader applications/application
3495 Sl   /usr/lib/x86_64-linux-gnu/unity-lens-files/unity-fil
3518 Sl   /usr/lib/x86_64-linux-gnu/unity-lens-music/unity-mus
3551 Ss   bash
3735 S    [kworker/u128:0]
3750 T    ./a.out
3796 S    [kworker/u128:2]
3816 S+   ./a.out
3818 R+   ps -e -o pid,stat,cmd
jgyan@ubuntu:~$
```

code explanation S+ single process which is the parent.

Unix daemons

Daemon: the term comes from Geek mythology, where daemons were guardian spirits.

- **Daemon**: Is a process that runs in the background and performs a specified operation at predefined times or in response to certain events.
  - Def: 2 – is a process that runs in the Background and is independent of control from all terminals.
  - There are **20-50** daemon processes running in a Unix system silently.
    There are several ways to start a daemon
    1. The system initialization scripts (/etc/rc)
    2. The inetd superserver
    3. Cron daemon : daemon that executes commands at a scheduled time.
    4. The at command
    5. From user terminals

- **Typical daemon process includes**: print spoolers, e-mail handlers, network systems and other programs that perform administrative tasks for the operating systems.

> **NB**: Almost all daemons have names that end with the letter "**d**". For example, httpd the daemon that handles the Apache server, or sshd (secure shell daemon,) which handles SSH remote access connections. **Linux often start daemons at boot time. Shell scripts stored in /etc/init.d directory are used to start and stop**

Functions of daemons

i. Open network port (such as port 80) and respond to network requests.
ii. Monitor systems such as hard disk health or RAID array.

How to find PID of the process?

The PID of the process can be found using "ps -ef command.

```
root@hardening:~# ps -ef
UID          PID  PPID  C STIME TTY          TIME CMD
root           1     0  1 11:15 ?        00:00:01 /sbin/init
root           2     0  0 11:15 ?        00:00:00 [kthreadd]
root           3     2  0 11:15 ?        00:00:00 [ksoftirqd/0
root           4     2  0 11:15 ?        00:00:00 [kworker/0:0
root           5     2  0 11:15 ?        00:00:00 [kworker/0:0
root           6     2  0 11:15 ?        00:00:01 [kworker/u2:
root           7     2  0 11:15 ?        00:00:00 [rcu_sched]
root           8     2  0 11:15 ?        00:00:00 [rcu_bh]
root           9     2  0 11:15 ?        00:00:00 [migration/0
root          10     2  0 11:15 ?        00:00:00 [watchdog/0]
root          11     2  0 11:15 ?        00:00:00 [kdevtmpfs]
root          12     2  0 11:15 ?        00:00:00 [netns]
root          13     2  0 11:15 ?        00:00:00 [perf]
root          14     2  0 11:15 ?        00:00:00 [khungtaskd]
root          15     2  0 11:15 ?        00:00:00 [writeback]
root          16     2  0 11:15 ?        00:00:00 [ksmd]
```

Coding Rules

- Some basic rules to coding a daemon prevent unwanted interactions from happening.
- We state these rules and then show a function, *daemonize,* that implements them.

1. The first thing to do is call umask to set the file mode creation mask to 0.
2. Call fork and have the parent exit.
3. Call setsid to create a new session.
4. Change the current working directory to the root directory.
5. Unneeded file descriptors should be closed.
6. Some daemons open file descriptors 0, 1, and 2 to /dev/null so that any library routines that try to read from standard input or write to standard output or standard error will have no effect.

Daemon process program in Linux environment

```
#include <stdio.h>
#include <unistd.h>
#include <syslog.h>
#include <fcntl.h>
void daemonize(const char *cmd)
{
        int i, fd0, fd1, fd2;
        pid_t pid;
        umask(0);
        if ((pid = fork()) < 0) {
        printf("%s: can't fork", cmd);
        exit(1);}
        else if (pid != 0) /* parent */
        exit(0);
        setsid();
        if (chdir("/") < 0) {
                perror("chdir: can't
                change directory to /");
                exit(1);
        }

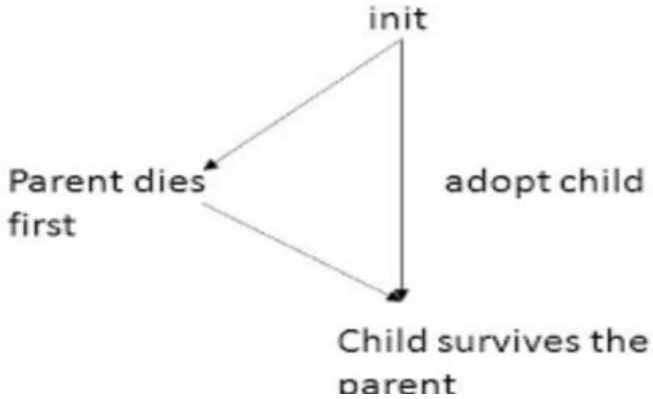
fd0 = open("/dev/null", O_RDWR);
fd1 = dup(0);
fd2 = dup(0);

exit(1);
}
```

Orphan process

- Created when a child process is running and the parent process is killed
- The system recognizes the orphaned child process.
- And passes the orphan process to init which then becomes the it's new parent process and terminates it.

- If a parent dies before its child process, the child is automatically adopted by the original init process.

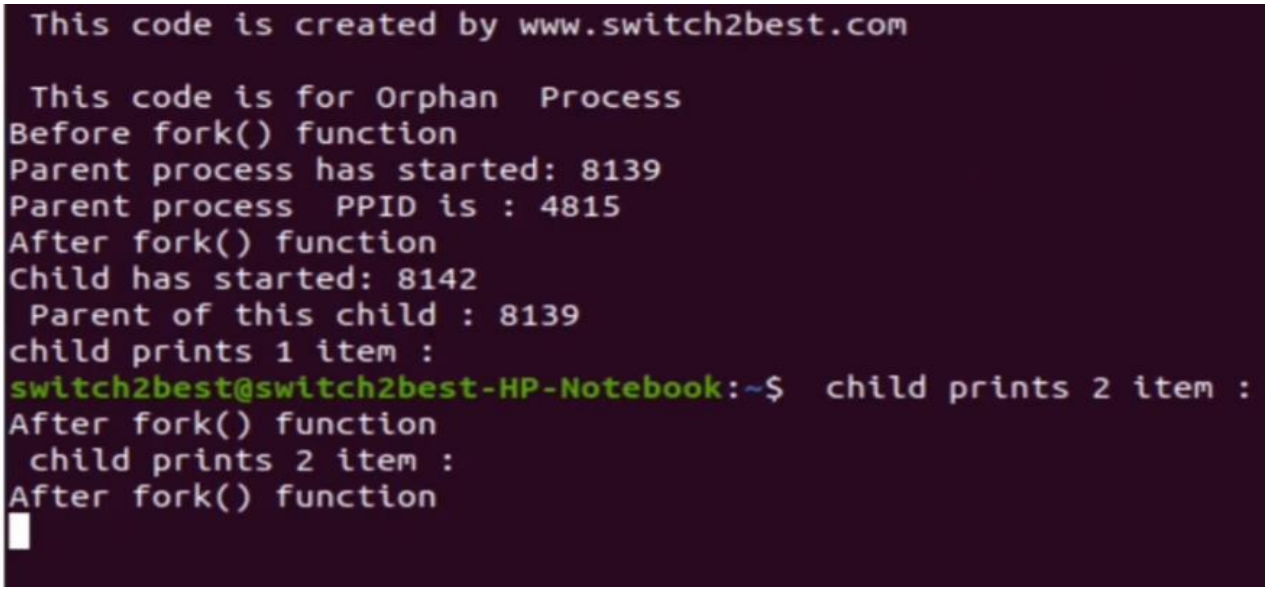


Write a C program to display the implementation of the orphan process

```
File  Edit  Format  View  Help
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
#include<sys/types.h>
#include<sys/stat.h>
#include<string.h>
printf("\n This code is created by www.switch2best.com\n");
printf("\n This code is for displaying orphan process\n");
int pid;
printf("Before fork () function\n");
pid=fork();
if(pid==0)
{
printf("child has started:%d\n",getpid ());
printf("parent of this child:%d \n",getppid());
printf("child print 1 item:\n");
sleep (30);
printf("child prints 2 item:\n");
}
else
{
printf("parent process has started:%d\n",getpid ());

printf("parent process PPID:%d\n",getppid ());
}
printf("After fork() function \n");
}
```

The output of orphan process



Revision Questions

1. What will be the output of the below C fork program assuming none of the fork fails.

```
#include<stdio.h>
#include<sys/types.h>
#include<unistd.h>
int main()
{
fork();
fork();
fork();
printf("hello\n");
return 0;
}
```

2. suppose that in the program the child process also needed to wait on the parent to complete some actions. What changes to the program would be required in order to enforce this?